COS 433 Problem Set 5

Due date: Wednesday, 12/2/25 at 11:59pm. See the course website for our submission and grading policies.

Proof citations. Your proofs may assume without proof any fact stated in lecture or stated on the problem sets. All other statements – including results from textbooks or other sources that were not mentioned in lecture – should be proved in your problem set write-up.

Grading Policy. A total score of 100 points constitutes full credit for the problem set. You are not expected to complete all of the problems. Feel free to work on whichever problems you prefer!

Problem 1: Remote Memory (40 points)

Alice has a huge amount of data, initially represented by a bit-string $x \in \{0,1\}^N$. Alice does not want to store all of her data locally; instead she wishes to hire Bob to store x on some remote server. However, Alice would like to be able to efficiently retrieve any individual bit x_i of memory at a later point in time by querying Bob.

Unfortunately, this poses a problem: if Alice does not store her memory locally, she has to worry that Bob may *lie* about the contents of her memory.

(a) (20 points) Assume the existence of a family of keyed collision-resistant hash functions mapping $\{0,1\}^{2\lambda} \to \{0,1\}^{\lambda}$. Design a cryptographic scheme supporting the following operations in polynomial time:

- $\mathsf{Store}(x) \to (\mathsf{st}_A, \mathsf{st}_B)$. Store is a randomized operation taking an input x and outputting a pair of internal states – one for Alice, and one for Bob.
- $\mathsf{Lookup}(\mathsf{st}_B, i) \to (b, \pi)$: this is an operation that Bob performs on his memory to look up the value of x_i, outputting a bit b along with a certificate π for the fact that $x_i = b$.
- $\mathsf{Verify}(\mathsf{st}_A, i, b, \pi) \to \{0,1\}$. This is an operation that Alice performs to check that the ith bit of x is indeed equal to b.

Prove that your scheme has the following properties:

1. **Correctness:** If Alice stores a string x with Bob and then Bob looks up location i, he will obtain (b, π) such that $b = x_i$ and $\mathsf{Verify}(\mathsf{st}_A, i, b, \pi) = 1$ with probability 1.

2. **Efficiency:** the size of honestly generated st_A is bounded by

$$|\mathsf{st}_A| \leq \mathsf{poly}(\lambda, \log N)$$

and similarly,

$$|\pi| \leq \mathsf{poly}(\lambda, \log N).$$

(However, $|\mathsf{st}_B|$ can be as large as $\mathsf{poly}(\lambda, N)$.)

3. **Security:** Fix a string x and index i, and let $(\mathsf{st}_A, \mathsf{st}_B) \leftarrow \mathsf{Store}(x)$. Suppose that an adversarial, polynomial-time Bob is given $(\mathsf{st}_A, \mathsf{st}_B)$. Then, the probability that Bob can find π such that $\mathsf{Verify}(\mathsf{st}_A, i, b = 1 - x_i, \pi) = 1$ is negligible in λ.

Hint: think about the Merkle tree construction from class.

(b) (20 points) What if Alice wants to *update* her memory after storing it with Bob? Show how to augment the scheme from part (a) with the following update procedure:

- $\mathsf{Update}(\mathsf{st}_B, i, b) \to (\mathsf{st}'_A, \mathsf{st}'_B, \pi)$ is an algorithm that takes as input a state st_B along with a write instruction ("write b into location i") and outputs a new state pair $(\mathsf{st}'_A, \mathsf{st}'_B)$ along with a certificate π.
- $\mathsf{VerifyUpdate}(\mathsf{st}_A, \mathsf{st}'_A, i, b, \pi) \to \{0, 1\}$ is an algorithm that takes as input an old state st_A, new state st'_A, update instruction (i, b), and certificate π.

Your construction should satisfy the following properties:

1. **Correctness:** After honestly initializing $(\mathsf{st}_A^{(0)}, \mathsf{st}_B^{(0)}) \leftarrow \mathsf{Store}(x)$ and then sequentially running an arbitrary number of update steps:
   - Each update $(\mathsf{st}_A, \mathsf{st}'_A, i, b, \pi)$ should be accepted by $\mathsf{VerifyUpdate}$ with probability 1.
   - Let the final state pair be $(\mathsf{st}^*_A, \mathsf{st}^*_B)$, and let x^* denote the string obtained from x by processing each update (i, b) honestly (directly on x). Then, the output of $\mathsf{Lookup}(\mathsf{st}^*_B, j)$ should be equal to (x^*_j, π) such that $\mathsf{Verify}(\mathsf{st}^*_A, j, x^*_j, \pi) = 1$.
2. **Efficiency:** the running time of Update should be $\mathsf{poly}(\lambda, \log N)$.[1]
3. **Security:** Let $\tilde{B}$ denote a polynomial-time adversarial Bob, and let $x, (i_1, b_1), \ldots, (i_t, b_t), i_{t+1}$ denote a fixed initial memory, update pattern, and lookup query. Finally, let $(\mathsf{st}_A^{(0)}, \mathsf{st}_B^{(0)}) \leftarrow \mathsf{Store}(x)$.

   Then, the probability that $\tilde{B}(\mathsf{st}_A^{(0)}, \mathsf{st}_B^{(0)})$ outputs updates $(\mathsf{st}_A^{(1)}, \pi_1, \ldots, \mathsf{st}_A^{(t)}, \pi_t, \pi_{t+1}$ such that:
   - $\mathsf{VerifyUpdate}(\mathsf{st}_A^{(k-1)}, \mathsf{st}_A^{(k)}, (i_k, b_k), \pi_k) = 1$ for all k, and
   - $\mathsf{Verify}(\mathsf{st}_A^{(t)}, 1 - x^*_{i_{t+1}}, \pi_{t+1}) = 1$.

   is negligible. In other words, $\tilde{B}$ cannot trick Alice into misreading any piece of her updated memory.

[1] In this problem, we assume that we can access an arbitrary memory location in constant time, which is the usual modeling assumption for arrays.

Problem 2: MACs and Random Oracles (30 points)

(a) (10 points) Show that the following scheme, defined relative to a hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$, is a secure MAC in the random oracle model.

- $\mathsf{Sign}^H(k, m) = H(k, m)$.
- $\mathsf{Verify}^H(k, m, \sigma) = 1$ if and only if $H(k, m) = \sigma$.

Hint: remember that the adversary can query both $\mathsf{Sign}^H(k, \cdot)$ as well as $H(\cdot, \cdot)$ on arbitrary inputs.

(b) (10 points) Next, we modify the scheme from part (a) slightly:

- $\widetilde{\mathsf{Sign}}^H(k, m)$ does the following:
  - Compute $v = H(k, m)$.
  - Set σ to be a description of a circuit $C(\cdot)$ such that $C(k') = v$ for all inputs k'. The circuit C ignores its input entirely and always outputs v.
- $\widetilde{\mathsf{Verify}}^H(k, m, \sigma)$ does the following:
  - Interpret σ as the description of a circuit C.
  - Check that $C(k) = H(k, m)$.

Prove that this scheme is *also* a (correct and) secure MAC in the random oracle model.

(c) (10 points) Prove that for *any polynomial-time computable function* $h : \{0,1\}^* \to \{0,1\}^\lambda$, the scheme $(\widetilde{\mathsf{Sign}}, \widetilde{\mathsf{Verify}})$ using hash function h is *insecure*.

This example *breaks* the random oracle heuristic, and indicates that security of a scheme in the random oracle model does not necessarily imply that there exists a secure real-world instantiation of the scheme.

Note: you can solve this without solving part (b).

Problem 3: Discrete Logarithm Proofs (20 points)

Let $G = \{G_\lambda\}_\lambda$ denote a family of effective groups of known prime size $|G_\lambda| = p_\lambda$ and known generators $\{g_\lambda\}$. Consider the following interactive protocol between a verifier that has a group element $h = g^x$ and a prover that has the exponent x.

- $P(g, x)$ samples a random exponent $r \leftarrow \mathbb{Z}_p$ and sends $R = g^r$ to the verifier.
- $V(g, h, R)$ samples a uniformly random exponent $s \leftarrow \mathbb{Z}_p$ and sends s to the prover.
- $P(g, x, r, s)$ returns $z = r + s \cdot x$.
- $V(g, h, R, z)$ checks that $g^z = R \cdot h^s$.

Observe that there is no nontrivial "language" in this protocol: every $h \in G$ has the property that there exists an x such that $g^x = h$.

Instead, prove the following fact: suppose a prover P^* is adversarial but does not have the discrete logarithm x as input. Nevertheless, suppose that $P^*(g, h)$ runs in polynomial time and causes the verifier V to accept with non-negligible probability. Using P^*, give a polynomial-time algorithm that recovers the discrete logarithm x from (g, h) with non-negligible probability.

Hint: Think about rewinding.

Note: This was the missing step in our proof of security of Schnorr signatures in Lecture 18!

Problem 4: Resources in Zero-Knowledge Proofs (40 points)

In this problem, we identify several features of zero-knowledge proofs that are *inherently necessary* (in the sense that zero-knowledge proofs without these features do not exist). To properly formulate this, we first describe a class of languages with trivial zero-knowledge proof systems.

(a) (10 points) We say that a language $L \subset \{0,1\}^*$ is in the class BPP if there exists a randomized polynomial-time algorithm "deciding membership in L," meaning that:

- On input $x \in L$, the algorithm outputs 1 with probability at least $2/3$.
- On input $x \notin L$, the algorithm outputs 0 with probability at least $2/3$.

Prove that every $L \in \mathsf{BPP}$ has a trivial, one-message zero knowledge proof system. You may allow the verifier a nonzero but negligible amount of "completeness error" (where the honest verifier accidentally rejects a true statement).

(b) (10 points) Let $L \subset \{0,1\}^*$ be any language, and suppose that L has a zero-knowledge proof system with *no interaction* (meaning that the verifier doesn't send the prover any messages). Prove that $L \in \mathsf{BPP}$.

(c) (10 points) Let $L \subset \{0,1\}^*$, and suppose that L has a zero-knowledge proof system with a *deterministic verifier*. Prove that $L \in \mathsf{BPP}$.

(d) (10 points) Let $L \subset \{0,1\}^*$, and suppose that L has a zero-knowledge proof system with a *deterministic prover*. Moreover, suppose the proof system satisfies **auxiliary-input zero knowledge:** for every polynomial-time $V^*(\cdot,\cdot)$, there exists a simulator $\mathrm{Sim}^*(\cdot,\cdot)$ such that for all $x \in L$ and *all polynomial-length auxiliary strings* z, we have

$$\mathrm{Sim}^*(x, z) \approx_c \mathrm{view}_{V^*}(x, z),$$

where $\mathrm{view}_{V^*}(x, z)$ is defined to be the view of $V^*(x, z)$ when interacting with the honest (computationally unbounded) prover P. This security definition asserts that a verifier V^* does not learn information from a proof *even if* V^* starts out with some extra knowledge z that the honest verifier does not have.

Under these conditions, prove that $L \in \mathsf{BPP}$.

Problem 5: Commitment Schemes (25 points)

For a one-way function f, let us define a non-interactive (unkeyed) commitment scheme $\mathsf{Com}_f(b; x, r) = (f(x), r, \langle x, r \rangle \oplus b)$.

(a) (5 points) Is Com_f both computationally hiding and statistically binding for all OWFs f?

(b) (10 points) Now suppose that f is an injective OWF. Is Com_f both computationally hiding and statistically binding for all such f?

(c) (10 points) Let $\mathcal{H} = \{\{H_k : \{0,1\}^{2\lambda} \to \{0,1\}^\lambda\}_{k \in \{0,1\}^\lambda}\}_\lambda$ be a family of collision-resistant hash functions with uniformly random keys $k \leftarrow \{0,1\}^\lambda$. We define a two message commitment scheme $\mathsf{Com}_{\mathcal{H}}(k, b; x, r) = (H_k(x), r, \langle x, r \rangle \oplus b)$. Prove that $\mathsf{Com}_{\mathcal{H}}$ is both computationally hiding and **computationally binding, defined below**.

A commitment scheme $\mathsf{Com}(\mathsf{ck}, m; r)$ is **computationally binding** if sending a commitment string y "effectively fixes the message m" for polynomial-time adversaries. Formally, we require that for every probabilistic polynomial-time adversary $\mathcal{A}$,

$$\Pr\Big[(m, r, m', r') \leftarrow \mathcal{A}(\mathsf{ck}) : m \neq m' \text{ and } \mathsf{Com}(\mathsf{ck}, m; r) = \mathsf{Com}(\mathsf{ck}, m'; r')\Big] \leq \mathsf{negl}(\lambda),$$

where the probability is taken over the choice of key ck and the internal randomness of $\mathcal{A}$.

Problem 6: Distinguishing Distributions (30 points)

For any polynomial-time program P and input length n, one can define an associated distribution D on output strings obtained by running P on a uniformly random $x \leftarrow \{0,1\}^n$.

In this problem, we consider the following computational task: given a pair of programs P_0, P_1 and an input length 1^n, decide whether:

- The statistical distance $\Delta(D_0, D_1) > 2/3$, or
- $\Delta(D_0, D_1) < 1/3$,

provided that one of the two is the case.

(a) (10 points) Under the quadratic residuosity assumption, prove that this computational task cannot be solved in polynomial time.

(b) (15 points) Your goal is to construct an Honest-Verifier Zero Knowledge proof system for this computational task. In this part, construct a **partial solution** with the following properties:

- The honest prover can be computationally unbounded,
- Completeness can hold with probability $1 - \mathsf{negl}(n)$ (the verifier may reject the honest prover with nonzero but negligible probability).
- Completeness and HVZK hold when $\Delta(D_0, D_1) > 1 - \mathsf{negl}(n)$, and

- Soundness holds when $\Delta(D_0, D_1) = \mathsf{negl}(n)$.

Prove that your protocol satisfies completeness, soundness, and HVZK. Your protocol should not rely on any computational assumptions.

(c) (5 points) Construct an HVZK proof system as in part (b) with improved parameters:

- Completeness and HVZK should now hold for $P_0, P_1, 1^n$ such that $\Delta(D_0, D_1) > 2/3$, and
- Soundness should now hold for $P_0, P_1, 1^n$ such that $\Delta(D_0, D_1) < 1/3$.